

# Convolutional and recurrent neural networks

Jan Skalski

30/04/2025

# Outline

- ① Convolutional neural networks
  - Motivation and advantages
  - Convolutional layer construction
  - Rationale - feature extraction
  - Practical aspects
- ② Recurrent neural networks
  - Motivation and advantages
  - Applications comparison
  - Simple RNN mechanism
  - LSTM and GRU models

## Why CNNs ?

To process data with more complex structure than vectors, especially grid-structured data such as images, where input elements exhibit strong local correlations.

## Advantages of CNNs

- Significant reduction in the number of parameters.
- Higher efficiency and less prone to overfitting.
- Capturing local spatial dependencies within the data.
- Translation invariance (any small shift in the input results only in the shift in the output in the respective direction).

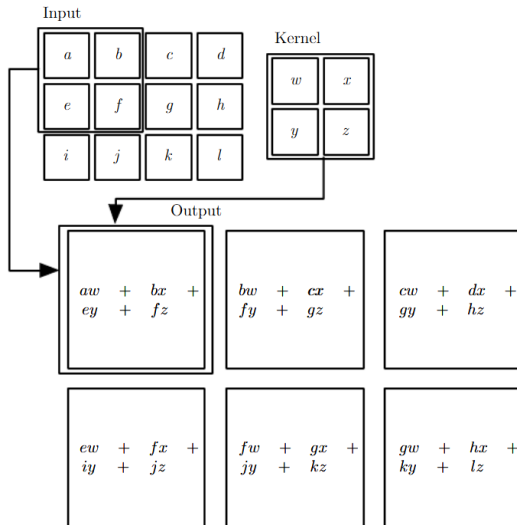
# Convolution operation

## Definition for matrices

Let  $W \in \mathbb{R}^{K_h \times K_v}$  and  $x \in \mathbb{R}^{M_h \times M_v}$ , where  $K_h \leq M_h$  and  $K_v \leq M_v$ . Then  $\star : \mathbb{R}^{K_h \times K_v} \times \mathbb{R}^{M_h \times M_v} \rightarrow \mathbb{R}^{(M_h - K_h + 1) \times (M_v - K_v + 1)}$  denotes the convolution operator between the matrix  $x$  and kernel  $W$ :

$$\left[ W \star x \right]_{i,j} = \sum_{n=0}^{K_h-1} \sum_{k=0}^{K_v-1} W_{K_h-n, K_v-k} x_{i+n, j+k},$$

for  $i = 1, 2, \dots, M_h - K_h + 1, j = 1, 2, \dots, M_v - K_v + 1$ .



Convolution operation illustration without kernel flipping [GBC16]

# Convolution operation

## Definition for tensors

Let  $W$  be a tensor of dimension  $(K_c \times K_h \times K_v)$  and  $x$  be a tensor of dimension  $(K_c \times M_h \times M_v)$ , where  $K_h \leq M_h$  and  $K_v \leq M_v$ . Then

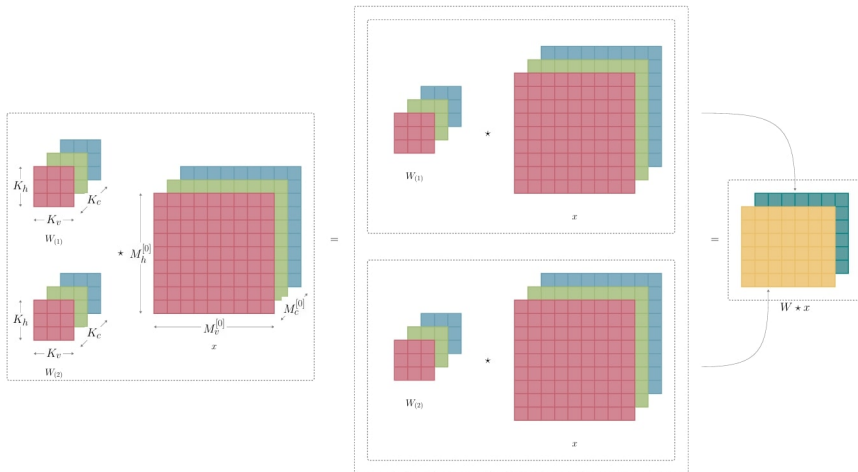
$$\star : \mathbb{R}^{K_c \times K_h \times K_v} \times \mathbb{R}^{K_c \times M_h \times M_v} \rightarrow \mathbb{R}^{(M_h - K_h + 1) \times (M_v - K_v + 1)}$$

denotes the convolution operator between the tensor  $x$  and filter  $W$ :

$$W \star x = \sum_{k=1}^{K_c} (W_k \star x_k),$$

where  $W_k$  and  $x_k$  are the  $k$ -th channels of  $W$  and  $x$ , respectively, and summation over matrices is element-wise.

# Convolutional layer construction



Layer with 3 channel input and 2 filters [LMN24]



- **Input** to the layer  $l$ :

$a^{(l-1)}$  - tensor of dimension  $(K_c^{(l-1)} \times M_h^{(l-1)} \times M_v^{(l-1)})$

- **Learnable parameters:**

$W^{(l),(k)}$  - filters of dimension  $(K_c^{(l-1)} \times K_h^{(l)} \times K_v^{(l)})$ ,  $k = 1, 2, \dots, K_c^{(l)}$

$$b^{(l)} = [b_1^{(l)}, b_2^{(l)}, \dots, b_{K_c^{(l)}}^{(l)}]$$

- **Convoluting** the  $a^{(l-1)}$  with filters:

$$z_k^{(l)} = \left( W^{(l),(k)} \star a^{(l-1)} \right) + b_k^{(l)}, k = 1, 2, \dots, K_c^{(l)}$$

- **Activating** tensors  $z^{(l)}$  (element-wise):

$$a^{(l)} = S^{(l)}(z^{(l)})$$

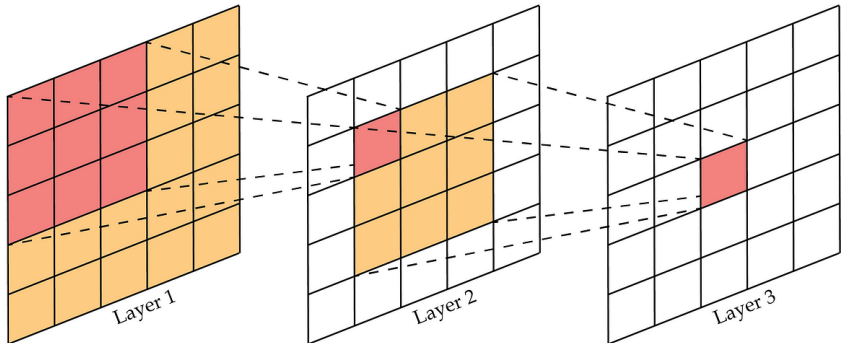
- **Output** of the layer  $l$ :

$a^{(l)}$  - tensor of dimension  $(K_c^{(l)} \times [M_h^{(l-1)} - K_h^{(l)} + 1] \times [M_v^{(l-1)} - K_v^{(l)} + 1])$

# Rationale - feature extraction

## How CNNs work in image processing?

- Filters extract input features in a hierarchical way throughout the hidden layers (receptive field of the output elements increases).
- Early layers capture low-level features of the image, such as edges, corners and simple textures.
- Deeper layers combine earlier features into more complex representations, enabling the model to answer higher-level questions about the image (e.g., determining whether a specific object is present or classifying the image).



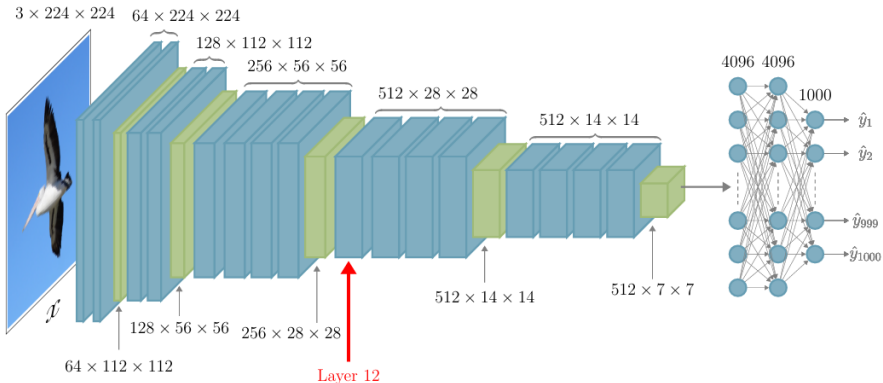
Effective receptive field increases along the deeper layers of the CNN

# Practical aspects

Convolutional layers usually include certain alterations:

- **padding** - adding extra zero-valued pixels around the input to retain same spatial dimensions between the input and output.
- **stride** - controlling the step with which the filter moves across the input (reducing the spatial dimension of the output).
- **dilation** - inserting zeros between the filter elements (increasing receptive field of the output).

In addition to convolutional layers, CNNs also incorporate **pooling** layers (to make deeper layers more **invariant to translations** in the input) and fully connected layers (to make final classifications based on the information aggregated by the convolutional layers).

$$3 \times 224 \times 224 \quad 64 \times 224 \times 224$$


VGG19 network structure with 24 layers: 16 convolutional, 5 pooling and 3 fully connected layers. **Input:** image of dimension 3x224x224. **Output:** 1000-element vector with class probabilities. **Total number of parameters:** 143, 667, 240. [LMN24]

# Recurrent Neural Networks

## Why RNNs?

To model sequential, dependent or temporal data, where past information influences future predictions, such as in text processing, speech recognition, and time series analysis.

## Advantages of RNNs

- Unlike feedforward networks (e.g., CNNs), the flow of information in RNNs has an internal feedback loop, capturing temporal dependencies in sequential data.
- The hidden state retains past information, influencing current outputs (memory-like property).
- Parameter sharing across the entire sequence, enhancing efficiency.
- Capable of processing inputs and generating outputs of variable length.

## Applications: CNNs vs RNNs

## CNNs - spatial data

- Image classification
- Image segmentation
- Object detection
- Style transfer

## RNNs - sequential data

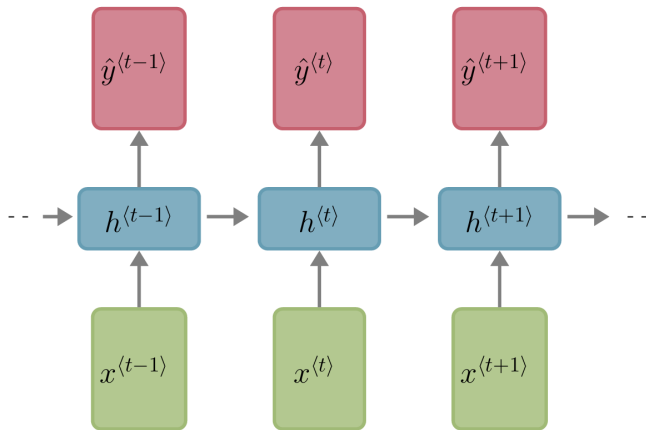
- **Text generation**
- **Machine translation**
- **Time series prediction**
- **Sentiment analysis**



## CNNs + RNNs

- **Speech recognition**
- **Facial identification**
- **Image captioning**
- **Video analysis**

# Simple RNN mechanism



Single layer recurrent network unrolling mechanism with input sequence in green, hidden state in blue and output sequence in red. [LMN24]

- **Input:**

$$x = (x^{[1]}, x^{[2]}, \dots, x^{[T]}), \text{ where every } x^{[t]} \in \mathbb{R}^p$$

- **Learnable parameters:**

$$W_h \in \mathbb{R}^{r \times r}, W_h^{(x)} \in \mathbb{R}^{r \times p}, W_h^{(y)} \in \mathbb{R}^{q \times r} \\ b_h \in \mathbb{R}^r, b_y \in \mathbb{R}^q$$

- **Updating hidden state**  $h^{[t]} \in \mathbb{R}^r$ :

$$h^{[t]} = S_h(W_h h^{[t-1]} + W_h^{(x)} x^{[t]} + b_h)$$

- **Activating hidden state:**

$$\hat{y}^{[t]} = S_y(W_h^{(y)} h^{[t]} + b_y)$$

- **Output:**

$$\hat{y} = (\hat{y}^{[1]}, \hat{y}^{[2]}, \dots, \hat{y}^{[T]}), \text{ where every } \hat{y}^{[t]} \in \mathbb{R}^q$$

# Long-Short Term Memory Layer

## Motivation

LSTM involves the separation of information flow into two distinct neuron groups:

- **hidden state** - carrying short-term dependencies,
- **cell state** - retaining long-term dependencies.

Hidden state and cell state neurons are connected via three internal gates: **input gate**, **forgetting gate**, and **output gate**, which are usually vectors within the range  $[0, 1]$ .

- Input:**

$$x = (x^{[1]}, x^{[2]}, \dots, x^{[T]}), \text{ where every } x^{[t]} \in \mathbb{R}^p$$

- Learnable parameters:**

$$W_x^{(f)} \in \mathbb{R}^{m \times p}, W_x^{(i)} \in \mathbb{R}^{m \times p}, W_x^{(o)} \in \mathbb{R}^{m \times p}, W_x^{(\tilde{c})} \in \mathbb{R}^{m \times p}$$

$$W_h^{(f)} \in \mathbb{R}^{m \times m}, W_h^{(i)} \in \mathbb{R}^{m \times m}, W_h^{(o)} \in \mathbb{R}^{m \times m}, W_h^{(\tilde{c})} \in \mathbb{R}^{m \times m}$$

$$b_f \in \mathbb{R}^m, b_i \in \mathbb{R}^m, b_o \in \mathbb{R}^m, b_{\tilde{c}} \in \mathbb{R}^m, b_y \in \mathbb{R}^q, W_h^{(y)} \in \mathbb{R}^{q \times m}$$

- Recurrent step:**

$$(c^{[t-1]}, h^{[t-1]}, x^{[t]}) \xrightarrow{\text{forgetting and input gates}} c^{[t]} \xrightarrow{\text{output gate}} h^{[t]}$$

- Activating hidden state:**

$$\hat{y}^{[t]} = S_y(W_h^{(y)} h^{[t]} + b_y)$$

- Output:**

$$\hat{y} = (\hat{y}^{[1]}, \hat{y}^{[2]}, \dots, \hat{y}^{[T]}), \text{ where every } \hat{y}^{[t]} \in \mathbb{R}^q$$

# Recurrent step in LSTM

- **Internal gates:**

$$g_f^{[t]} = \sigma(W_h^{(f)} h^{[t-1]} + W_x^{(f)} x^{[t]} + b_f) \quad \textbf{(forgetting gate)}$$

$$g_i^{[t]} = \sigma(W_h^{(i)} h^{[t-1]} + W_x^{(i)} x^{[t]} + b_i) \quad \textbf{(input gate)}$$

$$g_o^{[t]} = \sigma(W_h^{(o)} h^{[t-1]} + W_x^{(o)} x^{[t]} + b_o) \quad \textbf{(output gate)}$$

- **Internal cell state:**

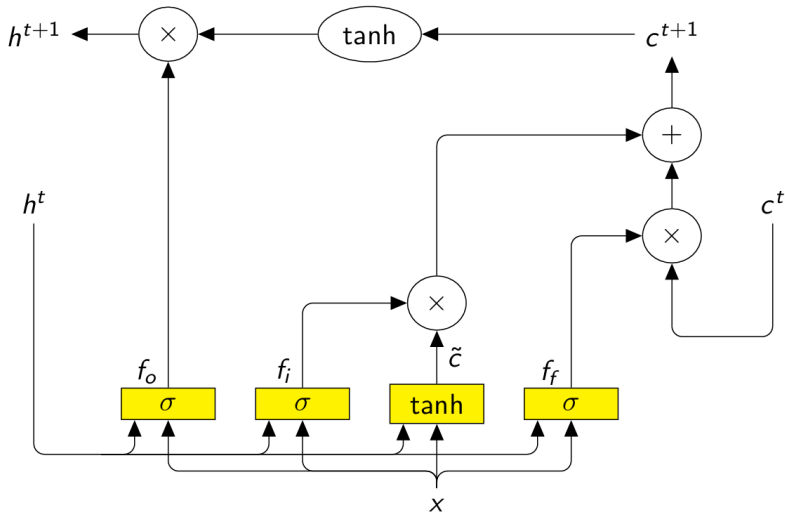
$$\tilde{c}^{[t]} = \tanh(W_h^{(c)} h^{[t-1]} + W_x^{(c)} x^{[t]} + b_c)$$

- **Cell state update:**

$$c^{[t]} = g_f^{[t]} \odot c^{[t-1]} + g_i^{[t]} \odot \tilde{c}^{[t]}$$

- **Hidden state update:**

$$h^{[t]} = g_o^{[t]} \odot \tanh(c^{[t]})$$



LSTM gate mechanism illustration [Jar]

# Gated Recurrent Unit

## Motivation

In the GRU model, both long-term dependencies and short-term memory are maintained within the **hidden state**. To achieve this, the hidden state is updated through two internal gating mechanisms: the **update gate** and the **reset gate**.



- **Input:**

$$x = (x^{[1]}, x^{[2]}, \dots, x^{[T]}), \text{ where every } x^{[t]} \in \mathbb{R}^p$$

- **Learnable parameters:**

$$W_x^{(r)} \in \mathbb{R}^{m \times p}, W_x^{(u)} \in \mathbb{R}^{m \times p}, W_x^{(\tilde{h})} \in \mathbb{R}^{m \times p}$$

$$W_h^{(r)} \in \mathbb{R}^{m \times m}, W_h^{(u)} \in \mathbb{R}^{m \times m}, W_h^{(\tilde{h})} \in \mathbb{R}^{m \times m}$$

$$b_r \in \mathbb{R}^m, b_u \in \mathbb{R}^m, b_{\tilde{h}} \in \mathbb{R}^m, b_y \in \mathbb{R}^q, W_h^{(y)} \in \mathbb{R}^{q \times m}$$

- **Recurrent step:**

$$(h^{[t-1]}, x^{[t]}) \xrightarrow{\text{reset and update gates}} h^{[t]}$$

- **Activating hidden state:**

$$\hat{y}^{[t]} = S_y(W_h^{(y)} h^{[t]} + b_y)$$

- **Output:**

$$\hat{y} = (\hat{y}^{[1]}, \hat{y}^{[2]}, \dots, \hat{y}^{[T]}), \text{ where every } \hat{y}^{[t]} \in \mathbb{R}^q$$

# Recurrent step in GRU

- **Internal gates:**

$$g_r^{[t]} = \sigma(W_h^{(r)} h^{[t-1]} + W_x^{(r)} x^{[t]} + b_r) \quad \textbf{(reset gate)}$$

$$g_u^{[t]} = \sigma(W_h^{(u)} h^{[t-1]} + W_x^{(u)} x^{[t]} + b_u) \quad \textbf{(update gate)}$$

- **Internal hidden state:**

$$\tilde{h}^{[t]} = \tanh(W_h^{(\tilde{h})} (g_r^{[t]} \odot h^{[t-1]}) + W_x^{(\tilde{h})} x^{[t]} + b_{\tilde{h}})$$

- **Hidden state update:**

$$h^{[t]} = (1 - g_u^{[t]}) \odot h^{[t-1]} + g_u^{[t]} \odot \tilde{h}^{[t]}$$

# References

- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville.  
*Deep Learning*. MIT Press, 2016.
- [Jar] Szymon Jaroszewicz. *Deep learning - slides from a lecture*.
- [LMN24] Benoit Liquet, Sarat Moka, and Yoni Nazarathy.  
*Mathematical Engineering of Deep Learning*. CRC Press, 2024.
- [Wan16] Tingwu Wang. *Recurrent Neural Networks*. 2016.